



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

Sección: 1

Tema 3 - Lenguajes y Gramáticas Formales

Profesor:

Felix Marquez

Alumnos:

Iván Hernández - C.I: 30.529.821

Miguel Gómez - C.I: 29.862.177

Miguel Barrios - C.I: 28.619.898

Daniel Valenzuela - C.I: 28.619.852

Puerto Ordaz, 17 de junio de 2026.

Introducción

El estudio de los lenguajes y gramáticas formales constituye la piedra angular de la informática moderna, trascendiendo la simple escritura de código para adentrarse en la fundamentación matemática de cómo las máquinas procesan la información. En el presente informe, se aborda el Tema 3 de la cátedra de Lenguajes y Compiladores, cuyo objetivo es dotar al ingeniero de las herramientas teóricas necesarias para modelar, validar y optimizar lenguajes de diversa índole.

A través de este desarrollo, exploraremos primero la Jerarquía de Chomsky, que clasifica las gramáticas según su poder expresivo y complejidad computacional, estableciendo los límites de lo que puede ser procesado por diferentes arquitecturas de cómputo. Posteriormente, aplicaremos estos conceptos al Modelado de Genoma, demostrando que la teoría de lenguajes permite representar estructuras biológicas complejas mediante Gramáticas Libres de Contexto.

Asimismo, se analiza la Higiene y Optimización de Gramáticas, un proceso crítico en la ingeniería de compiladores para eliminar patologías como la ambigüedad y la recursividad, asegurando que el análisis sintáctico sea determinista y eficiente. Finalmente, cerraremos con la implementación de Autómatas Finitos y Expresiones Regulares aplicados al estándar PGN de ajedrez, materializando la teoría en un motor de reconocimiento de patrones funcional. Este recorrido permite transformar la percepción del código fuente: de simple texto arbitrario a un objeto matemático con propiedades demostrables y robustas.

Fundamentos y Jerarquía de Chomsky

Un lenguaje formal es un sistema artificial diseñado bajo un estricto conjunto de normas preestablecidas para operar en entornos específicos. Estas reglas conforman la gramática del lenguaje. En lugar de palabras y oraciones espontáneas, se compone de un alfabeto de símbolos elementales que se agrupan en secuencias o cadenas de caracteres.

Frente a la creatividad y complejidad del lenguaje natural, que permiten su uso en las más diversas situaciones, los lenguajes formales han sido diseñados para ser utilizados en contextos muy precisos. Su campo de aplicación incluye la lógica, las matemáticas, la informática y la lingüística.

Backus Naur Form (BNF)

En la informática se utiliza algo llamado Backus Naur Form (BNF) que es una notación matemática o metalenguaje que se usa para describir de forma exacta y sin ambigüedades la sintaxis de un lenguaje, principalmente de los lenguajes de programación. O sea, todos los lenguajes de programación utilizan la notación BNF para definir sus reglas.

El BNF fue inventado por John Backus y Peter Naur en los años 60 para definir de forma rigurosa el lenguaje de programación ALGOL 60. Es importante conocer qué es el BNF para comprender los siguientes términos de mejor manera.

Sintaxis básica de BNF

La notación BNF utiliza un conjunto muy reducido de símbolos (metasímbolos) para construir sus reglas de producción:

- **::=**: Significa "**se define como**" o "**produce**". Separa el elemento que queremos definir (a la izquierda) de sus componentes (a la derecha).

- **< > (Llaves angulares):** Se usan para encerrar a los símbolos **No Terminales** (las variables o categorías estructurales del lenguaje). Por ejemplo: **<instruccion>**, **<digito>**.
- **" " o texto plano:** Los símbolos **Terminales** (los caracteres o palabras reales que escribe el programador) se escriben entre comillas o de forma literal para diferenciarlos. Por ejemplo: **"if"**, **"+"**, **"while"**.
- **| (Barra vertical):** Significa **"o"** (alternativa). Se usa para separar diferentes opciones válidas para una misma regla.

Ejemplo práctico para definir un número entero:

<entero> ::= <signo> <digito_no_cero> <resto_digitos> | <digito_no_cero> | "0"

<signo> ::= "+" | "-"

<digito> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Un **<entero>** se define como un **<signo>** seguido de un **<digito_no_cero>** y un **<resto_digitos>**, O simplemente un **<digito_no_cero>**, O el carácter literal **"0"**.

Un **<signo>** se define como el carácter literal **"+"** O el carácter literal **"-"**.

Los Cuatro Componentes de una Gramática Formal

Para comprender la clasificación de Chomsky debe tenerse en cuenta que una gramática formal está integrada por cuatro componentes:

- **Símbolos terminales (Σ).** Comprenden los elementos básicos del lenguaje (el alfabeto). Son los símbolos reales que el usuario escribe en su teclado y que el compilador lee al final. No se pueden cambiar por nada más. Ejemplo: Los números del 0 al 9, operadores + y -. Son los tokens puros que el analizador léxico reconoce.

- **Símbolos no terminales (N).** Elementos auxiliares o variables sintácticas que sirven como intermediarios durante el proceso de construcción, pero que nunca forman parte del resultado final. No forman parte del lenguaje. Son palabras o letras encerradas (en BNF se usan llaves angulares $\langle \rangle$) que sirven como marcadores de posición temporales. Te dicen qué *tipo* de estructura va en ese lugar, pero todavía tienen que transformarse. Al final del proceso, desaparecen por completo. Ejemplo: Dos etiquetas: $\langle \text{EXPRESIÓN} \rangle$ (para la operación completa) y $\langle \text{NÚMERO} \rangle$ (para los dígitos individuales).
- **Símbolo inicial o axioma (S).** El punto de partida obligatorio del cual se desglosan todas las combinaciones posibles del lenguaje. Ejemplo: El símbolo inicial para el ejemplo pasado podría ser la etiqueta $\langle \text{EXPRESIÓN} \rangle$. Siempre se empieza ahí porque el objetivo es construir expresiones matemáticas, en ese caso.
- **Conjunto de reglas de producción (P).** Son reglas que, partiendo del símbolo inicial, permiten realizar las transformaciones necesarias para obtener las palabras del lenguaje, mediante el reemplazo de los símbolos no terminales por símbolos terminales. En otras palabras, son las reglas del lenguaje. Tienen la forma $A ::= B$ (en BNF), lo que significa: "*Cuando detectes la etiqueta A, la puedes reemplazar por lo que está en B*".

Ejemplo de los cuatro componentes

$N = \{ \langle \text{EXPRESION} \rangle, \langle \text{NUMERO} \rangle \}$

$\Sigma = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, - \}$

$S = \langle \text{EXPRESION} \rangle$

$P = A ::= B$ (Las reglas)

$\langle \text{EXPRESIÓN} \rangle ::= \langle \text{NÚMERO} \rangle "+" \langle \text{NÚMERO} \rangle \mid \langle \text{NÚMERO} \rangle "-" \langle \text{NÚMERO} \rangle$

<NÚMERO> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

La Jerarquía de Chomsky: Clasificación de los Lenguajes

En un artículo publicado en 1956, el lingüista estadounidense Noam Chomsky (1928-) estableció cuatro tipos de lenguajes formales, en función de las gramáticas que los generan. Chomsky organizó estos lenguajes en cuatro niveles concéntricos (del Tipo 0 al Tipo 3), donde cada nivel impone más restricciones a sus reglas de producción que el anterior.

- **Tipo 0 - Lenguajes libres o recursivamente enumerables:** Responden a gramáticas libres. Las producciones pueden contener cadenas de terminales y no terminales tanto en el lado derecho como en el lado izquierdo.
 - **Restricción:** No hay restricciones en las reglas. El lado izquierdo solo requiere tener al menos un símbolo no terminal. Permiten que las cadenas se "encojan" o se eliminen símbolos (producciones vacías sin control).
 - **Automata que lo reconoce:** Maquina de Turing.
 - **Uso en la vida real:** No hay ningún lenguaje de programación comercial que sea Tipo 0. Para tratar de leer un lenguaje tipo 0 se usan las máquinas de Turing donde un programa podría quedarse en un bucle infinito intentando descifrar si una cadena es válida o no. Esto debido a su lógica destructiva y abstracta.
 - **Ejemplo:** *"Si juntas un bloque Azul, uno Rojo y uno Verde, los tres se fusionan y se convierten en un solo bloque Amarillo"*. Tres bloques desaparecieron para convertirse en uno solo. Hubo una contracción de elementos y el contexto de tenerlos juntos altera por completo la estructura. Las gramáticas de los lenguajes de programación normales (Tipo 2 o 3) no permiten esto; en ellas, una sola

variable se expande, nunca se encoge ni depende de combinaciones en el lado izquierdo.

- **Ejemplo en BNF:** El BNF estándar está diseñado para Tipo 2, pero para representar la lógica de Tipo 0 donde el contexto izquierdo/derecho altera la sustitución, se ilustra la dependencia de variables agrupadas:

$\langle S \rangle ::= \langle A \rangle \langle B \rangle$

$\langle A \rangle \langle B \rangle ::= c$; El contexto de tener A junto a B permite transformarlos en 'c'

$\langle A \rangle \langle B \rangle \langle C \rangle ::= "x"$

$\langle B \rangle \langle C \rangle ::= \langle C \rangle \langle B \rangle$

- **Tipo 1 - Lenguajes dependientes del contexto:** Aquellos definidos por gramáticas dependientes del contexto. Se reconocen porque permiten el reemplazo de símbolos no terminales solo en ciertos contextos. Las producciones de las gramáticas que generan estos lenguajes dependen del contexto en que aparece el símbolo no terminal y no reducen la longitud de la cadena. Se trata del tipo de lenguajes que poseen producciones más restringidas.
 - **Restricción:** Para cada regla la longitud del lado izquierdo debe ser menor o igual a la longitud del lado derecho. La única excepción es si la cadena está vacía. Básicamente, las cadenas no pueden "encogerse" durante la derivación. Un símbolo se transforma solo si está rodeado de ciertos vecinos.
 - **Autómata que lo reconoce:** Autómata Linealmente Acotado
 - **Uso en la vida real:** No se utiliza para diseñar lenguajes de programación comerciales, porque los algoritmos necesarios para analizar estas gramáticas son extremadamente lentos y complejos (requieren un espacio de memoria que crece proporcionalmente a la longitud de la cadena).

- **Ejemplo:** Las reglas de ortografía del español. La regla de la "M antes de la P y la B". Si tienes el sonido */N/*, solo puedes transformarlo en la letra "m" si a su derecha inmediata está la "p" (como en *campo*) o la "b" (como en *tambor*). Si a la derecha hay una "v", ese mismo sonido se transforma en "n" (como en *enviar*). Eso es, sensibilidad al contexto. El símbolo no se transforma libremente; depende de quién lo rodee.
- **Ejemplo en BNF:**

$$\langle S \rangle ::= a \langle A \rangle b$$

$$a \langle A \rangle ::= a a c \text{ ; El No Terminal } \langle A \rangle \text{ solo se transforma en 'a c' si tiene una 'a' a la izquierda}$$
- **Tipo 2 - Lenguajes independientes del contexto.** Aquellos que pueden ser reconocidos por una gramática libre de contexto. Las producciones de las gramáticas que dan lugar a estos lenguajes se atienen a la regla que establece que un símbolo no terminal puede ser siempre reemplazado por una cadena de símbolos no terminales o terminales sin considerar el contexto en el que se encuentre. Este tipo de lenguajes son los que más desarrollo han tenido, debido a que son la base de la mayoría de los lenguajes de programación.
 - **Restricción:** El lado izquierdo debe ser estrictamente un único símbolo no terminal. No importa qué símbolos rodeen a la variable, esta siempre se puede sustituir por lo que dice el lado derecho. Es el estándar para definir la sintaxis de los lenguajes de programación.
 - **Autómata que lo reconoce:** Autómata con Pila
 - **Uso en la vida real:** Se utiliza en el 100% de los lenguajes de programación comerciales. Es la herramienta matemática que define la estructura sintáctica de Python, Go, C++, Java y más lenguajes. Es una de las partes del compilador de

código, específicamente el analizador sintáctico, o sea el motor que toma las reglas BNF y verifica si el orden de los tokens es el correcto

- **Ejemplo:** Llenar un formulario en internet donde el campo diga `<FECHA_DE_NACIMIENTO>`, Al colocar la fecha, la etiqueta siempre se va a transformar en lo mismo: DD/MM/AAAA. Es libre de contexto.
- **Ejemplo de BNF:** BNF nació exclusivamente para el lenguaje tipo 2. Aquí no se fuerza la sintaxis como en el tipo 0 o tipo 1. Un ejemplo del Tipo 2 que los lenguajes Tipo 3 (regulares) no pueden resolver es el de los paréntesis balanceados (es decir, que por cada paréntesis que abras, se cierre uno obligatoriamente, permitiendo anidamientos infinitos como `((((( )))`).

`<S> ::= "(" <S> ")" | "a"`

(Este ejemplo genera paréntesis balanceados con una 'a' en el centro, como `(a)`, `((a))`, etc).

- **Tipo 3 - Lenguajes regulares o lineales.** La posición de un símbolo en una cadena depende únicamente del símbolo que lo precede. La calificación de regulares dada a este tipo de lenguajes obedece al hecho de que sus cadenas contienen regularidades o repeticiones de los mismos símbolos.
 - **Restricción:** Son las más restrictivas. El lado izquierdo debe ser un único no terminal. El lado derecho solo puede ser un terminal solo, o un terminal seguido de un no terminal (Regular Lineal Derecha). No se permiten anidamientos infinitos o memoria de conteo simultáneo.
 - **Autómata que lo reconoce:** Autómata Finito (AFD o AFND)
 - **Uso en la vida real:** Es una de las partes del compilador de código, específicamente el analizador léxico. Su trabajo es ir carácter por carácter

leyendo el código fuente para agruparlos en las palabras válidas del lenguaje. En la teoría, a estas palabras se les llaman Tokens o Símbolos Terminales.

- **Ejemplo:** La entrada de un estacionamiento inteligente que lee las placas de los carros. El sistema sabe que una placa válida debe tener obligatoriamente 3 letras seguidas de 4 números (por ejemplo, **ABC1234**). El sistema va leyendo de izquierda a derecha en línea recta. Si todo calza en ese orden lineal exacto, la acepta. No hay estructuras anidadas, no hay condiciones complejas, no hay memoria de retroceso. Es un proceso puramente secuencial.
- **Ejemplo en BNF:** Definir las reglas para identificar identificadores válidos en un lenguaje (nombres de variables que obligatoriamente deben empezar por una letra y luego pueden tener más letras o números). En BNF puro se vería así:

$$\langle S \rangle ::= "a" \langle Resto \rangle \mid "b" \langle Resto \rangle \mid "a" \mid "b"$$

$$\langle Resto \rangle ::= "a" \langle Resto \rangle \mid "b" \langle Resto \rangle \mid "1" \langle Resto \rangle \mid "2" \langle Resto \rangle \mid "a" \mid "b" \mid "1" \mid "2"$$

Punto 2 - Derivación y Modelado (Caso Practico: Dibujo)

En el ámbito de la ciencia de la computación y la informática teórica, los lenguajes formales constituyen el pilar fundamental para definir sintaxis, estructurar compiladores y automatizar procesos lógicos. La Jerarquía de Chomsky clasifica estas gramáticas en cuatro niveles distintos según su poder de reconocimiento y restricciones estructurales. Entre ellas, las Gramáticas Libres de Contexto (GLC) o de Tipo 2 ocupan un lugar privilegiado debido a su equilibrio óptimo entre expresividad matemática y eficiencia computacional.

Más allá de su aplicación tradicional en el análisis sintáctico de lenguajes de programación, las GLC poseen una notable capacidad para el modelado procedural y la generación de gráficos abstractos. Mediante la asignación de semánticas geométricas a los

símbolos terminales, un flujo puramente textual derivado de una gramática puede transformarse en estructuras visuales complejas, tales como fractales, polígonos o ramificaciones biológicas (Sistemas-L). Este informe detalla exhaustivamente los conceptos de derivación formal, la integración del modelo geométrico de la tortuga y la arquitectura del intérprete desarrollado en Python para materializar dicho proceso.

Una Gramática Libre de Contexto se define formalmente como una cuádrupla algebraica $(G = (V, \Sigma, P, S))$, donde (V) representa el conjunto de variables o símbolos no terminales, (Σ) es el alfabeto de símbolos terminales, (P) es el conjunto finito de reglas de producción y (S) es el símbolo inicial perteneciente a (V) . La característica definitoria de una gramática de Tipo 2 es que el lado izquierdo de cualquier producción en (P) consiste estrictamente en un único símbolo no terminal, lo que implica que la sustitución de dicha variable puede realizarse independientemente del contexto físico que la rodee.

La **derivación** es el proceso secuencial mediante el cual se aplican las reglas de producción para transformar el símbolo inicial (S) en una cadena compuesta exclusivamente por elementos de (Σ) . Cuando existen múltiples variables en una cadena intermedia (forma sentencial), se adopta una estrategia sistemática para garantizar la consistencia. En la *derivación por la izquierda* (leftmost derivation), se expande rigurosamente la variable que se encuentra más a la izquierda en cada paso de sustitución. Este método no solo facilita el análisis sintáctico subyacente, sino que determina el orden cronológico exacto en que la herramienta de dibujo procesará las instrucciones espaciales.

Modelado Geométrico y Gráficos de Tortuga

La traducción de cadenas abstractas a representaciones bidimensionales se fundamenta en los conceptos de la Geometría de la Tortuga (Turtle Graphics), introducida originalmente por Seymour Papert, y los Sistemas de Lindenmayer (Sistemas-L). A diferencia de la geometría cartesiana tradicional, que depende de coordenadas globales absolutas $((x,$

y)), la geometría de la tortuga opera bajo un paradigma de estado relativo local. El agente de dibujo (la tortuga) posee un estado instantáneo definido por su posición actual y su orientación o ángulo de dirección.

Para modelar estructuras ramificadas o anidadas sin perder la continuidad del trazo, se requiere un mecanismo que rompa la linealidad del dibujo. Esto se logra mediante la implementación de una estructura de datos de tipo **Pila (Stack)** basada en operaciones LIFO (Last In, First Out). Ciertos símbolos terminales actúan como comandos de control de contexto: uno de ellos empuja (push) el estado actual de la tortuga a la pila, aislando un punto de bifurcación, mientras que otro extrae (pop) dicho estado, forzando un retorno instantáneo al origen de la rama sin alterar el lienzo durante el trayecto.

Especificación Práctica de la Gramática Diseñada

Con el objetivo de cumplir con la actividad propuesta sobre el alfabeto $\Sigma = \{a, c, g, t\}$, se estructuró una gramática que organiza los comandos de dibujo en bloques atómicos no ambiguos. La distribución conceptual y semántica de los símbolos terminales se detalla en la siguiente tabla:

Símbolo Terminal	Semántica de Operación	Acción en la Tortuga
a	Avanzar	Desplaza la tortuga hacia adelante una distancia prefijada emitiendo un trazo visible.
g	Girar	Modifica la orientación rotando un ángulo preestablecido a la derecha
Símbolo Terminal	Semántica de Operación	Acción en la Tortuga

		de su vector actual.
c	Comenzar Rama	Almacena la tupla de estado (posición, orientación) en el tope de la pila de memoria.
t	Terminar Rama	Recupera el último estado de la pila; la tortuga salta a esa posición levantando el lápiz.

Las reglas de producción operan bajo una recursividad por la derecha para concatenar las instrucciones de dibujo ($(D \rightarrow T \mid D \mid T)$), garantizando que cualquier flujo semántico sea evaluable de izquierda a derecha. El símbolo no terminal (T) evalúa comandos simples ((a, g)) o encierra sub-bloques de dibujo recursivos mediante la estructura simétrica de control ($c \mid D \mid t$).

Arquitectura e Implementación del Código en Python

La implementación práctica requiere mapear la gramática formal a código imperativo. El módulo estándar turtle de Python es idóneo para este propósito, complementado por una lista nativa que opera como pila de ejecución. A continuación, se detalla el bloque lógico desarrollado:

```
import turtle
def dibujar_cadena(instrucciones, longitud_trazo=50,
angulo_giro=90):
    """
    Intérprete procedural para procesar cadenas generadas
    por la GLC definida sobre el alfabeto {a, c, g, t}.
    """
    pila_estados = []

    # Inicialización del entorno gráfico
    pantalla = turtle.Screen()
    pantalla.title(f"Intérprete de Gramática - Cadena: {instrucciones}")
```

```

        trazador =
turtle.Turtle()
trazador.pensize(2)
trazador.speed(3) # Flujo
controlado para inspección
visual

        # Procesamiento secuencial (Lectura de la
Cadena) for simbolo in instrucciones: if
simbolo == 'a':
        trazador.forward(longitud_trazo)
        elif
simbolo == 'g':
        trazador.right(angulo_giro)
        elif
simbolo == 'c':
        # Operación PUSH: Almacena el estado espacial actual
estado_actual = (trazador.position(), trazador.heading())
pila_estados.append(estado_actual)
        elif
simbolo == 't':
        # Operación POP: Retorna al último punto de bifurcación
if pila_estados:
        posicion_guardada, angulo_guardado = pila_estados.pop()
trazador.penup() # Desactiva el trazo durante el retorno
trazador.goto(posicion_guardada)
trazador.setheading(angulo_guardado) trazador.pendown() #
Reactiva el dibujo para nuevos trazos

elif simbolo == ' ':
        continue else: print(f"Advertencia:
Símbolo '{simbolo}' fuera del alfabeto funcional.")

        pantalla.exitonclick()

```

Análisis modular del software: La función principal encapsula el estado local de la tortuga mediante el objeto trazador. El bucle iterativo actúa de manera análoga a una Unidad de Control en una arquitectura computacional, decodificando secuencialmente cada carácter. Las instrucciones críticas son c y t: el uso de trazador.position() captura las coordenadas cartesianas flotantes mientras que trazador.heading() extrae el ángulo actual en grados, salvaguardándolos en pila_estados mediante el método nativo .append(). Al encontrarse el

token de terminación, `.pop()` remueve dicho elemento del tope, y los métodos de control de lápiz (`penup()`/`pendown()`) garantizan el principio de invarianza del entorno espacial no ramificado.

Evaluación Metódica de las Derivaciones de Prueba

Para validar la corrección semántica de la gramática y el correcto funcionamiento del intérprete, se analizaron cinco casos geométricos derivados paso a paso:

- **Caso 1 (Línea Base):** Expresado como la cadena física "a". Representa la traza elemental. La gramática deriva directamente $(S \rightarrow D \rightarrow T \rightarrow a)$. El programa se limita a avanzar la distancia asignada en línea recta.
- **Caso 2 (Cuadrado Regular):** Cadena "a g a g a g a g". Al configurar el parámetro `angulo_giro=90`, el intérprete concatena cuatro trazos ortogonales idénticos que regresan cíclicamente al punto de origen del plano, delimitando un polígono cerrado simétrico de cuatro aristas.
- **Caso 3 (Estructura de Retorno Vacío):** Cadena "c a t". Demuestra la preservación del estado. La tortuga guarda su origen (c), genera un trazo hacia el frente (a) y retrocede de manera invisible (t) al origen absoluto.
- **Caso 4 (Estructura de Bifurcación Arbórea):** Cadena "a c g a t a". Si se parametriza con un ángulo de 45° , este caso simula el crecimiento de la flora artificial. La tortuga genera un tronco vertical básico, guarda el nodo intermedio, rota de forma oblicua y dibuja una rama lateral secundaria. Posteriormente, regresa al nodo central y continúa con el eje de crecimiento principal hacia arriba.
- **Caso 5 (Esquina Isométrica):** Cadena "a g a c a g t". Representa la generación de gráficos tridimensionales proyectados en entornos bidimensionales. El programa traza una arista de base, efectúa una rotación y abre una rama de perspectiva para trazar la profundidad matemática de un vértice del cubo sin corromper el flujo secuencial del contorno general.

Punto 3 - Higiene y Optimización de Gramáticas

Las gramáticas formales pueden sufrir patologías que impiden la construcción de un analizador sintáctico correcto y eficiente.

Ambigüedad

Definición

Una gramática G es ambigua si existe al menos una cadena $w \in L(G)$ que admite dos o más árboles de derivación distintos. El compilador no puede determinar cuál estructura sintáctica es la correcta, lo que produce código objeto incorrecto.

Gramática ambigua de ejemplo

Se utiliza la gramática de expresiones aritméticas sin jerarquía de operadores:

$G = (\{E\}, \{+, *, id\}, P, E)$

P :

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

Demostración: dos árboles para la cadena $id + id * id$

La misma cadena $id + id * id$ puede derivarse de dos formas distintas, cada una generando un árbol diferente:

Árbol 1 — suma evaluada primero

Derivación:

E

|— E → id

|— +

└— E

|— E → id

|— *

└— E → id

Resultado: id + (id * id)

La multiplicación tiene precedencia implícita.

Árbol 2 — multiplicación evaluada primero

Derivación:

E

|— E

| |— E → id

| |— +

| └— E → id

|— *

└— E → id

Resultado: (id + id) * id

La suma tiene precedencia implícita.

Solución: estratificar la gramática por precedencia

Se introduce un no terminal por cada nivel de precedencia. Los operadores de menor precedencia quedan en los niveles superiores de la jerarquía.

$G' = (\{E, T, F\}, \{+, *, (,), id\}, P', E)$

P' :

$E \rightarrow E + T \mid T \quad \leftarrow \text{precedencia baja (+)}$

$T \rightarrow T * F \mid F \quad \leftarrow \text{precedencia media (*)}$

$F \rightarrow (E) \mid id \quad \leftarrow \text{átomo}$

Resultado

G' es no ambigua. La cadena $id + id * id$ tiene ahora un único árbol de derivación: $E \Rightarrow E + T \Rightarrow T + T * F \Rightarrow F + F * F \Rightarrow id + id * id$, garantizando que la multiplicación siempre tiene mayor precedencia que la suma.

Recursividad por la Izquierda

Definición

Una gramática tiene recursividad por la izquierda cuando existe un no terminal A tal que $A \Rightarrow^+ A\alpha$ para alguna cadena α . Esto causa que los parsers de descenso recursivo (LL) entren en un bucle infinito al intentar expandir A , pues lo primero que hacen es volver a llamar a A .

Gramática con recursión izquierda directa

La siguiente gramática para expresiones aditivas presenta recursión izquierda directa:

P :

$E \rightarrow E + T \quad \leftarrow \text{recursión izquierda: } E \text{ aparece al inicio}$

$E \rightarrow T$

$T \rightarrow id$

Problema: bucle infinito en parser LL

Para derivar E, el parser aplica $E \rightarrow E + T$; para derivar ese E, vuelve a aplicar $E \rightarrow E + T$; y así indefinidamente.

Algoritmo de eliminación — paso a paso

Para una producción de la forma $A \rightarrow A\alpha \mid \beta$ (donde β no comienza por A), el algoritmo es:

Paso	Acción
1	Identificar la producción recursiva izquierda: $E \rightarrow E + T$. Se extrae $\alpha = "+ T"$
2	Identificar la producción base (no recursiva): $E \rightarrow T$. Se extrae $\beta = "T"$
3	Crear un nuevo no terminal E' (llamado también "E primo")
4	Reescribir: $A \rightarrow \beta A'$ y $A' \rightarrow \alpha A' \mid \varepsilon$

Gramática resultante

P' (sin recursión izquierda):

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \varepsilon$$

$$T \rightarrow id$$

Verificación

Derivación de la cadena $id + id + id$ con la gramática corregida:

E

$$\Rightarrow T E'$$

$$\Rightarrow id E'$$

$$\Rightarrow id + T E'$$

$$\Rightarrow id + id E'$$

$$\Rightarrow id + id + T E'$$

$$\Rightarrow id + id + id E'$$

$$\Rightarrow id + id + id \varepsilon$$

$$= id + id + id \quad \checkmark$$

Resultado

La gramática resultante es equivalente a la original (genera el mismo lenguaje) pero no tiene recursión izquierda, lo que la hace apta para parsers LL(1) y de descenso recursivo. E' acumula los términos adicionales sin llamarse a sí misma al inicio.

Factorización por la Izquierda

Definición

Una gramática requiere factorización por la izquierda cuando dos o más producciones para el mismo no terminal comparten un prefijo común. En ese caso, un parser LL(1) no puede decidir cuál producción aplicar con un solo símbolo de anticipación (lookahead), ya que el prefijo no permite distinguir entre las alternativas.

Caso de estudio: el dangling else

El ejemplo canónico es la sentencia if-then-else en lenguajes de programación:

G: $S \rightarrow \text{if } E \text{ then } S \text{ else } S \quad \leftarrow \text{producción 1}$

$S \rightarrow \text{if } E \text{ then } S \quad \leftarrow \text{producción 2}$

Problema: prefijo común "if E then S"

Al leer el token "if", el parser no puede saber si la sentencia tiene cláusula else o no. Para saberlo tendría que leer toda la sub-sentencia completa, violando la restricción de lookahead = 1 de los parsers LL(1).

Proceso de factorización paso a paso

Paso	Acción
1	Identificar el prefijo común: "if E then S"
2	Identificar los sufijos de cada alternativa: "else S" (producción 1) y ϵ (producción 2, no tiene sufijo)
3	Crear un nuevo no terminal S' que contenga los sufijos como alternativas
4	Reemplazar las producciones originales por la versión factorizada

Gramática resultante — optimizada

G' (factorizada):

$S \rightarrow \text{if } E \text{ then } S S'$

$S' \rightarrow \text{else } S \mid \varepsilon$

Cómo el parser decide ahora con un solo lookahead

Token de anticipación	Producción aplicada	Significado
else	$S' \rightarrow \text{else } S$	La sentencia tiene rama alternativa
Cualquier otro token	$S' \rightarrow \varepsilon$	La sentencia no tiene else; se expande a vacío

Resultado

La gramática G' es equivalente a G y puede procesarse con un parser LL(1). El símbolo S' resuelve la ambigüedad del dangling else: un else siempre se asocia al if más cercano, gracias a que $S' \rightarrow \text{else } S$ tiene prioridad sobre $S' \rightarrow \varepsilon$ cuando el parser ve el token "else".

Resumen Comparativo

Patología	Síntoma	Técnica	Resultado
Ambigüedad	Múltiples árboles para la misma cadena	Estratificación por precedencia	Gramática no ambigua, un único árbol
Recursividad izquierda	Parser LL entra en bucle infinito	Introducción de no terminal A'	Gramática apta para LL(1)
Prefijo común (factorización)	Parser no puede elegir producción con 1 lookahead	Factorización + nuevo no terminal	Parser LL(1) decide con un solo token

Punto 4 - DE la Expresión al Autómata (Caso Práctico: Ajedrez)

De la Expresión al Autómata (Caso Práctico: Ajedrez)

El desarrollo de software enfocado en el análisis de lenguajes (ya sean de programación o de notaciones específicas como el ajedrez) requiere la traducción de reglas semánticas humanas a estructuras matemáticas que una computadora pueda procesar en tiempo lineal.

Para este caso práctico, se modela un procesador capaz de leer cadenas de caracteres y determinar si corresponden a una jugada válida en el tablero de ajedrez bajo el estándar de almacenamiento de partidas PGN.

Notación PGN (Portable Game Notation): Planteamiento Teórico y Ejecución

La Notación de Juego Portable (PGN) es el estándar internacional para registrar partidas de ajedrez en un formato de texto plano estructurado. En su estado completo, el lenguaje PGN es un lenguaje libre de contexto debido a la necesidad de validar reglas complejas (como si una pieza realmente puede moverse a la casilla indicada según la posición actual del tablero).

Sin embargo, **la sintaxis estructural de un movimiento aislado** puede ser clasificada y validada como un **Lenguaje Regular**. Esto nos permite utilizar la herramienta más eficiente de la teoría de la computación para su reconocimiento: un **Autómata Finito Determinístico (AFD)**.

Ejecución Formal del Autómata

Un AFD ejecuta el reconocimiento de una cadena mediante una función de transición extendida, denotada como $\delta^*(q, w)$, donde q es el estado actual y w es la cadena restante. El autómata lee un símbolo a la vez de izquierda a derecha.

- Si al consumir toda la cadena el autómata se encuentra en un **Estado de Aceptación** ($q_f \in F$), la cadena se declara **Válida**.
- Si el autómata procesa un símbolo que no tiene transición válida, cae en un **Estado Trampa o de Error** (q_e), rechazando inmediatamente la cadena.

Definición de un Subconjunto Simplificado del PGN

Para garantizar la viabilidad del diseño y concentrarnos en la mecánica pura de las expresiones regulares y los autómatas, se define un **subconjunto simplificado y estricto** que abarca exclusivamente los movimientos simples de piezas y peones hacia casillas vacías.

Restricciones del Alfabeto (Σ)

El alfabeto total de nuestro lenguaje está compuesto exclusivamente por los siguientes subconjuntos de caracteres ASCII:

1. **Identificadores de Piezas (P):** Representados por una letra mayúscula correspondiente al nombre de la pieza en inglés: K (Rey), Q (Dama), R (Torre), B (Alfil), N (Caballo).
2. **Identificadores de Columnas (C):** Una letra minúscula que define la coordenada horizontal del tablero: {a, b, c, d, e, f, g, h}.
3. **Identificadores de Filas (F):** Un dígito numérico que define la coordenada vertical del tablero: {1, 2, 3, 4, 5, 6, 7, 8}.

Reglas Gramaticales del Subconjunto

Cualquier cadena válida debe cumplir con la siguiente estructura secuencial:

- **Opcional:** Una única letra del conjunto de Piezas (P). Si se omite, la gramática asume por defecto que el movimiento pertenece a un **Peón**.
- **Obligatorio:** Una única letra del conjunto de Columnas (C).
- **Obligatorio:** Un único número del conjunto de Filas (F).

Diseño de la Expresión Regular (Regex) y el Autómata Finito Determinístico (AFD)

1. Expresión Regular (Regex)

La expresión regular formal que describe este lenguaje algebraico es:

Regex = $^{\wedge}[KQRBN]?[a-h][1-8]^{\$}$

- $^{\wedge}$ y $^{\$}$: Representan las anclas de inicio y fin de línea, asegurando que no existan caracteres basura antes o después del token.
- **[KQRBN]?**: El cuantificador '?' define opcionalidad (0 o 1 ocurrencia de los caracteres dentro del corchete).
- **[a-h]**: Obliga la presencia de exactamente un carácter en el rango de columnas.
- **[1-8]**: Obliga la presencia de exactamente un carácter en el rango de filas.

2. Definición Formal del AFD

El autómata se define matemáticamente como una 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, donde:

- $Q = \{q_0, q_1, q_2, q_f, q_e\}$ (Conjunto finito de estados)
- $\Sigma = P \cup C \cup F$ (Alfabeto completo)
- $q_0 = q_0$ (Estado inicial)
- $F = \{q_f\}$ (Conjunto de estados de aceptación)
- δ : Función de transición descrita por la siguiente matriz:

Estado Actual	Carácter leído: Pieza (P)	Carácter leído: Columna (C)	Carácter leído: Fila (F)	Cualquier Otro Carácter
q0	q1	q2	qe	qe

q1	qe	q2	qe	qe
q2	qe	qe	qf	qe
qf	qe	qe	qe	qe
qe	qe	qe	qe	qe

Funcionamiento del Programa en C++

La implementación del autómata en código utiliza el paradigma de **Mapeo Directo de Transición de Estados**. A continuación, se detalla minuciosamente cómo opera la lógica interna del software desarrollado:

1. Arquitectura Lógica

El programa no depende de librerías externas ni de motores complejos de expresiones regulares basados en backtracking, lo que garantiza un rendimiento óptimo de $O(N)$ en tiempo y $O(1)$ en memoria. Se estructura mediante:

- **Un tipo enumerado (enum Estado):** Que mapea uno a uno los estados matemáticos del diseño (q0, q1, etc.), forzando al compilador a validar únicamente los estados existentes.
- **Predicados Booleanos:** Funciones de un solo ciclo de reloj (esPieza, esColumna, esFila) que verifican por rangos ASCII si el carácter pertenece a una clase del alfabeto.
- **Estructura de Control Principal (switch-case anidada):** Funciona como el motor de inferencia del autómata. El switch externo evalúa el estado en el que se encuentra la

máquina, y las condiciones internas evalúan el carácter de entrada para determinar el siguiente estado.

2. Flujo de Ejecución Paso a Paso (Rastreo Lógico)

Cuando se invoca la función encargada de procesar la cadena, el ciclo de vida del procesamiento sigue los siguientes pasos:

- **Inicialización:** La variable de estado se setea en el estado inicial (q_0).
- **Ciclo de Lectura:** Se inicia un bucle que recorre la cadena índice por índice, extrayendo cada carácter individualmente.
- **Evaluación de Transición:**

Si se lee un carácter, el programa busca la sección correspondiente al estado actual dentro del bloque de decisiones.

Se evalúa a qué subconjunto del alfabeto pertenece el carácter. Si coincide con una regla válida de la matriz de transición, la máquina cambia al nuevo estado.

 - Si el carácter no es esperado en ese estado específico, el flujo cae en una rama residual de descarte y asigna el estado de error (q_e).
- **Optimización por Falla (Early Exit):** Inmediatamente después de evaluar el carácter, el programa verifica si se ha caído en el estado de error (q_e). De ser así, se rompe el ciclo y se interrumpe la simulación. Esto previene procesar caracteres innecesarios una vez que ya se detectó un fallo sintáctico irreparable.
- **Veredicto:** Una vez consumida por completo la cadena (o interrumpida por error), se realiza la evaluación lógica de cierre. Si la cadena se quedó corta (por ejemplo, leyendo solo una pieza como "N") o si sobraron caracteres tras una jugada correcta (como "Nf3f"), el estado final no coincidirá con el estado de aceptación (q_f), marcando la jugada como inválida.

Conclusión

La realización de este trabajo de investigación y desarrollo permite concluir que la comprensión de las gramáticas formales es indispensable para el diseño de software robusto y eficiente. A través de la resolución de los puntos planteados, se han obtenido las siguientes certezas técnicas:

La Jerarquía de Chomsky no es solo teórica: Dicta la viabilidad técnica de un proyecto. Entender que un analizador léxico requiere un autómata finito (Tipo 3) mientras que uno sintáctico necesita una gramática libre de contexto (Tipo 2) es vital para elegir la herramienta adecuada y evitar el uso excesivo de recursos computacionales.

El modelado formal es universal: La aplicación de gramáticas al genoma humano evidencia que cualquier sistema basado en reglas y secuencias —sea biológico o digital— puede ser abstraído y analizado bajo los mismos principios computacionales, permitiendo la creación de algoritmos de derivación precisos.

La "higiene" gramatical es un requisito de ingeniería: Se demostró que una gramática mal diseñada (ambigua o con recursividad izquierda) es una fuente directa de errores en un compilador. La aplicación de algoritmos de factorización y eliminación de recursividad es lo que separa un prototipo académico de una herramienta de producción estable (LL/LR).

Los autómatas son el puente entre la abstracción y el código: El ejercicio práctico con el formato PGN de ajedrez reafirma que las expresiones regulares y los AFD son el mecanismo más eficiente para el reconocimiento de patrones, siendo la base fundamental para el procesamiento de cualquier protocolo de datos moderno.

En definitiva, el dominio de estos fundamentos permite al ingeniero en informática dejar de ser un simple consumidor de lenguajes para convertirse en un arquitecto capaz de manipular los cimientos de la computación, garantizando la precisión, neutralidad y eficiencia en el procesamiento de cualquier lenguaje formal.

Referencias bibliográficas

Fuentes Bibliográficas Iván Hernández

- **Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007).** *Introducción a la Teoría de Autómatas, Lenguajes y Computación* (3ra ed.). Pearson Educación.
- **Chomsky, N. (1956).** *Three models for the description of language*. IRE Transactions on Information Theory, 2(3), 113-124.
- **Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008).** *Compiladores: Principios, Técnicas y Herramientas* (2da ed.). Pearson Educación. (Conocido en la ingeniería como "El libro del dragón").
- **Kelley, D. (1995).** *Teoría de Autómatas y Lenguajes Formales*. Prentice Hall.
- **Backus, J. W. (1959).** *The syntax and semantics of the proposed international algebraic language of the Zurich ACM-GAMM Conference*. Proceedings of the International Conference on Information Processing, UNESCO, 125-132.
- **Naur, P. (Ed.). (1960).** *Report on the algorithmic language ALGOL 60*. Communications of the ACM, 3(5), 299-314.
- **Farías, Gilberto (17 de septiembre de 2025).** Lenguajes formales. Enciclopedia Concepto. Recuperado el 13 de junio de 2026 de <https://concepto.de/lenguajes-formales/>.
- **Leiva, G. (2022).** *Lenguajes Formales y Computabilidad: Resumen Módulo 1 - Fundamento de los lenguajes formales*. Scribd. <https://es.scribd.com/document/585606427/Lenguajes-Formales-y-Computabilidad-Resumen-Modulo-1>

Fuentes Bibliográficas Daniel Valenzuela

- **Aho, A. V., Lam, M. S., Sethi, R. & Ullman, J. D. (2007).** *Compilers: Principles, Techniques, and Tools* (2ª ed.). Pearson.
- **Hopcroft, J. E., Motwani, R. & Ullman, J. D. (2006).** *Introduction to Automata Theory, Languages, and Computation* (3ª ed.). Pearson.
- **Chomsky, N. (1956).** *Three models for the description of language*. IRE Transactions on Information Theory, 2(3), 113–124.

Fuentes Bibliográficas Miguel Gomez

- **Chomsky, N. (1956).** *Three models for the description of language*. IRE Transactions on Information Theory, 2(3), 113-124.
- **Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007).** *Introducción a la teoría de autómatas, lenguajes y computación* (3a ed.). Pearson Educación.
- **Papert, S. (1980).** *Mindstorms: Children, computers, and powerful ideas*. Basic Books.

- **Prusinkiewicz, P., & Lindenmayer, A.** (1990). *The Algorithmic Beauty of Plants*. Springer-Verlag.

Fuentes Bibliográficas Miguel Barrios

- **Hopcroft, J. E., Motwani, R., & Ullman, J. D.** (2007). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Addison-Wesley. (Fundamentos de equivalencias entre expresiones regulares y diseño de autómatas finitos determinísticos).
- **Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D.** (2008). *Compiladores: Principios, Técnicas y Herramientas* (2da ed.). Pearson Educación. (Estrategias de traducción de tablas de transición a estructuras Switch-Case de alta eficiencia).
- **Edwards, S. J.** (1994). *Standard: Portable Game Notation Specification and Implementation Guide*. US Chess Federation. (Especificación técnica de los tokens lexicográficos para coordenadas y piezas en computación aplicada al ajedrez).